

Does Per-Token Routing Help Tiny Hybrid Language Models? A Budget-Matched Two-Corpus Study and a Normalization Audit

Chiheb Nabil

Remote Skills LTD

chiheb.design@gmail.com

Abstract

Hybrid language models that blend neural networks with n -gram statistics conventionally use a few fixed, corpus-level weights (Mikolov et al., 2011; Sundermeyer et al., 2012). We test whether replacing every fixed weight with a *learned per-token router* helps in the small-footprint regime that on-device applications occupy, routing among six frozen experts (a Kneser-Ney trigram, two recency bigrams, a 512-token cache, a continuation unigram, and a 3.3–8.2M-parameter GRU) on WikiText-2 and the Penn Treebank under one aligned-scoring protocol. We report two results. First, a methodological one that applies to any gated mixture: our initial version of this study found a 27–33% perplexity win for the router, and the win was an artifact of the evaluation, not of the model. The router’s “vote” features were the experts’ probabilities *of the word being scored*, so its weights depended on the candidate; the resulting mixture summed to 2.66 (WikiText-2) and 2.56 (PTB) over the vocabulary instead of 1, and every router perplexity was deflated by that factor. A one-line vocabulary-mass audit exposes this, and we release it as a guard. Second, the corrected result: with strictly causal features and a *budget-matched* protocol (router, static mixture and interpolation weight all tuned on validation only), a 15.5 KB router buys 13.2% (WikiText-2) and 5.3% (PTB) perplexity over tuned two-expert interpolation, but only 1.1% and 2.2% over a tuned static mixture of the same six experts — and the same router trained on the training stream is *far worse* than both (250.8 and 138.2), because expert reliability is not stationary between the stream the experts were built on and held-out text. The tuned static hybrid itself is the real win at this footprint: 119.1 and 89.7 at 68.7 and 28.7 MB, better than the GRU alone (146.9/103.0) and than a KenLM 5-gram (250.9/138.4). All code, logs, and the invalidated artifacts are released.

1 Introduction

Mixture architectures with learned routing are among the oldest ideas in neural computation (Jacobs et al., 1991; Jordan and Jacobs, 1994) and now scale to frontier models (Shazeer et al., 2017). We claim no architectural novelty. We ask a question the literature has not answered at the *bottom* of the footprint range: when every expert is a frozen classical statistic plus one small recurrent network, does learned per-token routing beat the tuned fixed interpolation that practitioners actually deploy?

The fixed-weight answer is settled at larger scales: interpolating a neural LM with Kneser-Ney smoothing using one or a few heldout-tuned weights has been standard practice for over a decade (Mikolov et al., 2011; Sundermeyer et al., 2012; Chang et al., 2015). At those scales the neural component dominates, so the blending scheme is a detail. In the small-footprint regime—on-device, offline, CPU-only personalization (Mathur et al., 2023)—the blending scheme is one of the few levers left, and nobody has checked whether per-token routing changes the conclusion there.

We ran that experiment twice. The first run said yes, emphatically: a 13.5 KB router appeared to cut perplexity by 33% (WikiText-2) and 27% (PTB) relative to tuned fixed interpolation, at every seed, on two corpora. The result was wrong, and the way it was wrong is the first contribution of this paper (§4). The router’s input included, for each expert, the probability that expert assigned to the *gold* next word. Those features were literally the labels: $X[:, :5]$ was bit-identical to $L[:, :5]$ in the collected dataset. Because the weights therefore depended on the word being scored, the “mixture” was not a probability distribution — summing it over the vocabulary gave 2.66 (WikiText-2) and 2.56 (PTB) instead of 1.000, while every baseline row summed to 1.000.

Router perplexities were deflated by exactly that factor; normalized, the same trained router scores ≈ 240 and ≈ 174 , i.e. *worse* than the tuned interpolation it appeared to beat by a third, and worse than the GRU alone.

§4 turns that failure into a cheap, reusable audit. The rest of the paper reports the corrected experiment, in which the router sees only causal, candidate-independent evidence: each expert’s confidence in its own favourite word, whether the five favourites agree, and 14 context statistics.

Findings. On two corpora under one aligned protocol, with every tuned component given the same budget (validation data only):

- **Candidate-conditioned gating is not a language model.** Its vocabulary mass is >1 , its perplexity is deflated by that mass, and the “hindsight oracle” it is usually compared to is deflated too. A sum-to-one check (§4) catches the whole class of bug in one line.
- **Per-token routing helps a little, not a lot.** A valid-tuned 15.5 KB router reaches 117.9 (WT-2) and 87.8 (PTB), versus 135.8/92.7 for tuned two-expert interpolation ($-13.2\%/ -5.3\%$) and 119.1/89.7 for a tuned static mixture of the same six experts ($-1.1\%/ -2.2\%$; three seeds, sd 0.06 on PTB).
- **Training the router on train is a trap.** The same architecture trained on the $10\text{--}13\times$ larger training stream scores 250.8 (WT-2) and 138.2 (PTB) — worse than uniform mixing (182.6/138.4) and worse than every tuned static baseline. Expert reliability shifts sharply between the stream the statistical experts were built on and held-out text (KN3: $34.7 \rightarrow 165.1$ PP from PTB train to test), and a train-fit router memorizes that shift (§6).
- **The hybrid, not the routing, is the win.** A tuned static mixture of five cheap experts and a small GRU reaches 119.1 (WT-2) / 89.7 (PTB) at 68.7/28.7 MB — better than the GRU alone (146.9/103.0), than tuned two-expert interpolation (135.8/92.7 is close on PTB) and than a toolkit KenLM 5-gram (250.9/138.4).

We release the full stack—every expert, the router, the aligned protocol, the audit scripts, the seed audits, and the *invalidated* v1 artifacts with

the code that produced them—so that both the result and the mistake are checkable.¹

2 Related Work

Interpolated language models. Linear interpolation of probability estimates originates with [Jelinek \(1980\)](#) and was systematized by Kneser-Ney smoothing ([Kneser and Ney, 1995](#)) and its empirical study ([Chen and Goodman, 1999](#)). Our trigram expert follows this line; our interpolation *weights* are the object of study.

Fixed-weight neural/ n -gram hybrids. Neural probabilistic LMs ([Bengio et al., 2003](#)), and RNN and LSTM LMs in particular, have long been interpolated with Kneser-Ney models using heldout-tuned fixed weights ([Mikolov et al., 2010, 2011](#); [Sundermeyer et al., 2012](#); [Chang et al. \(2015\)](#) discriminatively train a handful of context-dependent scaling factors—the closest classical relative of our router, at ASR scale and with features that do not include the scored word’s own probabilities. Our baselines are exactly this family.

Mixture of experts. Gating networks date to [Jacobs et al. \(1991\)](#) and [Jordan and Jacobs \(1994\)](#); sparse routing at scale to [Shazeer et al. \(2017\)](#). That lineage learns routers for *trainable* experts at footprints orders of magnitude above ours. A gate that reads the candidate’s own expert scores is unobjectionable in discriminative reranking of an n -best list, where the score is never required to be a normalized distribution; the failure mode we document is carrying that construction into perplexity.

Prediction with expert advice. Online bounds against the best expert in hindsight ([Freund and Schapire, 1997](#); [Cesa-Bianchi and Lugosi, 2006](#)) motivate the per-position-best-expert diagnostic. We stress that this diagnostic is *not* a realizable bound for a mixture LM (§4).

3 System and Protocol

Experts. Six experts, all frozen after fitting: (1) a modified Kneser-Ney trigram (discount scale 1.15, shared across corpora); (2–3) lag-2 and lag-3 recency bigrams; (4) a 512-token adaptive

¹<https://github.com/chihebnabil/hedge>. The system is named HEDGE in the repository after the prediction-with-expert-advice lineage ([Freund and Schapire, 1997](#)); the router itself is a supervised MLP, not multiplicative weights.

cache; (5) a continuation unigram; (6) a word-level GRU (vocabulary +1; 3.3M parameters on PTB, 8.2M on WT-2; Adam, cosine schedule, best-by-validation checkpoint; batchified perplexity 86.3 on PTB, 119.2 on WT-2).

Router. A one-hidden-layer MLP with a softmax over the six experts. Its 24 input features are a function of the *context only*: five expert confidences $\log p_i(\hat{w}_i(\text{ctx}))$ where $\hat{w}_i$ is expert i ’s own favourite word, five agreement flags (does expert i ’s favourite equal the plurality favourite), and 14 context statistics (Kneser-Ney backoff mass and continuation counts, recency-bigram totals and hit flags, two age-invariant cache densities, a 4-way backoff-bucket one-hot, position in sentence, sentence-start flag). Hidden size $h \in \{24, 64, 128, 256\}$ gives a 2.9–31.0 KB router. Training minimizes the mixture’s negative log-likelihood by Adam over collected aligned features.

The invariant. Because the weights $\alpha(\text{ctx})$ do not depend on the scored word, $P(w \mid \text{ctx}) = \sum_i \alpha_i p_i(w \mid \text{ctx})$ sums to 1 over the vocabulary whenever each expert does (unit-tested). Every perplexity we report is a perplexity.

Aligned scoring. All systems score *one shared gold token stream* per corpus at exactly the positions the router sees ($\sum_s \text{len}(s) - 1$ per split), so every row of every table is comparable by construction. Sentence-initial targets and `eos` are excluded; this makes our numbers systematically *higher* than batchified literature numbers for the same model, and we therefore never compare against published perplexities directly.

Budget-matched tuning. Every tuned component gets validation data and nothing else: the interpolation weight λ^* of the classical two-expert hybrid ($p = \lambda p_{\text{KN3}} + (1 - \lambda) p_{\text{GRU}}$; λ is therefore the *trigram* weight) from a 201-point grid on validation ($\lambda^* = 0.21$ on WT-2, 0.28 on PTB — the trigram earns a larger share on the smaller corpus); the static mixture α by EM/gradient on validation; the router’s hidden size by 2-fold cross-validation *inside* validation and its weights by fitting validation. Fitting on the training stream is reported separately as a negative control, because the training stream is the corpus the statistical experts were built from.

	WT-2	PTB
train tokens	1,927,034	907,138
valid tokens	201,797	71,825
test tokens	226,731	80,594
aligned test positions	217,004	75,623
vocabulary (from train)	28,714	9,644

Table 1: Corpora, as actually scored. Both use the same pipeline: our tokenizer, vocabulary from train, valid/test OOV mapped to `unk`; WikiText-2 additionally drops `= header = lines` and lowercases, so its token counts are below the commonly quoted 2.0M/214k/245k. PTB is the Mikolov split.

4 The Audit: Candidate-Conditioned Gating Is Not an LM

A gated mixture is a distribution only if its weights are a function of the context. If the gate reads anything about the word w being scored, then

$$Z(\text{ctx}) = \sum_{w \in V} \sum_i \alpha_i(x_{\text{ctx},w}) p_i(w \mid \text{ctx}) \neq 1, \quad (1)$$

and $\exp(-\frac{1}{N} \sum \log P(w_t))$ is not a perplexity: it is deflated by roughly $\exp(\overline{\log Z})$. Nothing in the training loop signals this — the objective decreases, the validation curve looks healthy, and the model can even “beat” a hindsight oracle, because the oracle $\max_i p_i(w_t)$ has the same defect.

Measurement. We enumerate Z over the full vocabulary at randomly sampled aligned test positions, with the cache streamed causally so its state matches evaluation (`benchmarks/normalization_audit.py`). On the v1 leaky router: $Z = 2.66$ (WT-2) and 2.56 (PTB); the per-position best-expert “oracle” reaches $Z \approx 3.1$. On KN3, the GRU and the fixed-interpolation baseline: $Z = 1.000$. On the corrected causal router: $Z = 1.000$ by construction and by measurement.

Consequence. Table 2 restates the v1 headline. The deflation factor is almost exactly the size of the claimed win, which is why the artifact survived three seeds, two corpora, an ensemble and a size sweep: all of them inherited the same features.

The guard. Two checks now run in the test suite and in the regeneration pipeline: (i) `gate_hybrid.load_split` refuses to start if any feature column of the collected dataset is bit-identical to any label column; (ii) `tests/test_gate.py` asserts that two

router	reported	mass Z	$\times Z$
WT-2, v1 (leaky)	90.3	2.66	≈ 240
PTB, v1 (leaky)	68.0	2.56	≈ 174
WT-2, corrected	117.9	1.000	117.9
PTB, corrected	87.8	1.000	87.8
all baselines	—	1.000	—

Table 2: The v1 artifact. “Reported” is the full-test perplexity the protocol printed; Z is the mean vocabulary mass measured at 100–150 sampled aligned positions with the cache streamed causally; $\times Z$ is what the reported number becomes once each position is normalized by its own Z . On the same samples the audit prints the pair directly (WT-2 v1: 120.2 $\rightarrow$ 319.5). Baseline rows measure $Z = 1.000$, which is why they were comparable to each other and not to the router. “All baselines” = KN3, the GRU, fixed- λ and static- α .

streams sharing a context but differing afterwards produce *identical* feature rows, that each expert sums to 1 over the vocabulary, and hence that any context-only mixture does too. This is the same discipline that caught an earlier Witten-Bell baseline whose distribution summed to >1 and whose perplexity was deflated by it; sum-to-one is not a nicety, it is the precondition for comparing rows.

5 Results

The router wins, barely (Table 3). Against the classical two-expert baseline and with an identical tuning budget, per-token routing over all six experts cuts perplexity by 13.2% (WT-2, 117.9 vs. 135.8) and 5.3% (PTB, 87.8 vs. 92.7) relative to tuned two-expert interpolation — but by only 1.1% (117.9 vs. 119.1) and 2.2% (87.8 vs. 89.7) relative to a tuned static mixture of the same six experts. On PTB the gap to static mixing is 1.94 PP against a router seed sd of 0.06, so it is real but small; on WT-2 it is 1.28 PP against a seed sd of 0.29. The honest reading is that most of the gain over classical interpolation comes from *having six experts and tuning their global weights on held-out data*, not from reweighting them per token: the static mixture alone recovers 171.0 $\rightarrow$ 119.1 on WT-2 and 101.7 $\rightarrow$ 89.7 on PTB, and the router adds 1–2% on top.

Without the GRU, routing does help — over uniform. Restricting the router to the five statistical experts (valid-tuned) gives 194.5 (WT-2) and 132.8 (PTB), against 263.9 and 196.2 for uniform mixing of the same five. So the router learns something real about when to trust a trigram over

a cache or a recency bigram; it is just that a tuned global mixture already captures most of the available gain once a strong neural expert is in the pool.

Router size does not matter. Sweeping $h \in \{24, 64, 128, 256\}$ for the train-trained router (WT-2 test 263.6/254.2/263.7/250.8; PTB 141.2/138.2/138.3/144.8) gives a flat, noisy curve whose 13 PP spread on WT-2 exceeds every effect we claim. For the valid-tuned router, 2-fold CV on validation is flat too (WT-2 CV valid 122.42/122.37/**122.08**/122.16; PTB 92.71/92.52/**92.18**/98.27) and picks $h = 128$ (15.5 KB) on both corpora. An earlier version of this work reported corpus-dependent optima (13.5 KB on WT-2, 2.6 KB on PTB); that curve was a property of the leaky features, not of the corpora.

Seeds. The valid-tuned router is stable: WT-2 118.06 ± 0.29 (ensemble 117.60) and PTB 87.84 ± 0.06 (ensemble 87.74) over three seeds, against static baselines that are deterministic to two decimals (119.14 ± 0.01 , 89.71 ± 0.00). The router’s margin over static mixing, seed-mean to seed-mean, is 1.08 PP (≈ 3.7 router-sd) on WT-2 and 1.87 PP (≈ 31 sd) on PTB. The train-trained negative control is reported at a single seed; its instability is the point, and the four-size sweep is its spread.

Selective expert activation does not survive the correction. The router’s GRU weight is computed from context alone, so thresholding it is a free, causal skip rule, and under the v1 protocol it appeared to run the GRU on 47–55% of tokens for ≤ 0.5 PP. With the corrected router the GRU carries most of the weight nearly everywhere: even at $\tau = 0.5$ it is still consulted on 99.0% (WT-2) and 89.7% (PTB) of tokens, at 118.1 and 89.9 PP. The cost lever was an artifact of the same leak.

6 Why Routing Does Not Transfer

Table 4 isolates the failure. Two facts matter. First, the statistical experts are not merely weaker on held-out text, they are *differently* weak: the trigram degrades $4.8\times$ from its own training stream while the GRU degrades $2.2\times$, so the trust allocation that is optimal on train is wrong on test. Second, the train-trained router’s *mean* weights, frozen and applied statically, score 105.4 on PTB test versus the router’s own 138.2 per-context decision: the average is fine, the modulation is not.

	WT-2 PP	WT-2 MB	PTB PP	PTB MB
pure KN3 trigram	296.2	21.4	165.2	9.2
KenLM 5-gram (toolkit defaults)	250.9	53.8	138.4	23.1
pure GRU (aligned)	146.9	32.7	103.0	13.1
uniform mixture (6 experts)	182.6	68.7	138.4	28.7
fixed-weight KN3+GRU (λ^* on valid)	135.8	54.0	92.7	22.3
static mixture, 6 experts, fit on train	171.0	68.7	101.7	28.7
static mixture, 6 experts, fit on valid	119.1	68.7	89.7	28.7
router, train-trained (negative control)	250.8	68.7	138.2	28.7
router, valid-tuned ($h=128$ by CV)	117.9	68.7	87.8	28.7
3-seed mean $\pm$ sd	118.1 $\pm$ 0.3		87.84 $\pm$ 0.06	
3-router ensemble	117.6		87.74	
static- α valid-fit, 3-seed sd	$\pm$ 0.01		$\pm$ 0.00	
router, 5 stat. experts only (valid-tuned)	194.5	36.0	132.8	15.6
router, 5 stat. experts only (train-trained)	481.0	36.0	226.7	15.6
best single expert, hindsight (normalized)	146.9	—	103.0	—
per-position best expert (<i>unnormalized</i>)	(57.4)	—	(44.4)	—
<i>v1 leaky router (invalidated)</i>	90.3		68.0	

Table 3: Main results: word-level test perplexity under one aligned protocol (217,004 WT-2 / 75,623 PTB positions), every tuned component selected on validation only. Footprints are per corpus and measured (`footprint_meta.py`): statistical experts 36.0/15.6 MB, GRU 32.7/13.1 MB, router 2.9–31.0 KB (15.5 KB at the selected $h = 128$). The router is the only row whose weights vary per token; it wins, but by 1.1%/2.2% over a tuned static mixture of the same six experts, and the two-expert interpolation row uses only KN3 and the GRU. The last row is the invalidated v1 protocol (§4), kept here so the correction is visible.

Validation tuning sidesteps the whole problem, which is why the budget-matched protocol is not a formality here — it is the difference between a router that helps (87.8) and one that costs 57% more perplexity than it should on PTB (138.2) — and 113% more on WT-2 (250.8 vs. 117.9).

Cost. One torch thread, 200 test sentences, 7 interleaved rounds, median quoted; the harness now asserts that no system measures faster than a component it contains — the check the v1 cost table failed, having timed the GRU alone at 5.09 ms/token and the hybrid containing it at 3.60. WT-2, in ms/token: GRU alone 4.21, two-expert interpolation 4.26, the same six experts mixed statically 4.32, routed hybrid 4.89. Routing therefore costs +0.57 ms/token in situ (+13.2%), and 0.44 ms/token measured in isolation (features + MLP, no GRU) = 9.0% of the hybrid. On PTB, where the GRU is 5 $\times$ cheaper, the same rows are 0.78 / 0.85 / 0.88 / 1.46: routing costs +0.58 ms/token (+66%) and 0.31 ms/token in isolation (21% of the hybrid). Evaluating the four extra experts is nearly free (+0.04 to +0.06 ms/token); the router is not. The KN3 top-1 memo is cleared every round, so these rows are worst-case for routing. Energy is a wall-time $\times$ 15 W TDP proxy, not measured power. Read with the quality rows: routing costs 13–66% more per token than statically

mixing the same experts, for 1–2% perplexity.

7 Limitations

(1) *Scale*: WT-2 and PTB are small corpora and our absolute numbers are not comparable to published ones (our protocol excludes sentence-initial targets and `eos`; batchified evaluation of the same GRU gives 119.2 / 86.3). We claim only the controlled comparison. (2) *Feature set*: our causal features are confidences, agreement flags and count statistics. A router given richer causal evidence — expert top- k identities as embeddings, per-expert entropies — might extract more; we did not test that, and at this footprint the flat size sweep suggests features, not capacity, are the binding constraint. (3) *Experts*: all six are frozen; joint training changes the reliability-shift story of §6 and is the obvious next step. (4) *Baselines*: no transformer expert, one tokenizer, two English corpora, CPU timing on one machine. (5) *Effect size*: the router’s advantage over a tuned static mixture (2.2% on PTB, 1.1% on WT-2) is small enough that seed variance matters: three seeds give 118.1 $\pm$ 0.3 (WT-2) and 87.84 $\pm$ 0.06 (PTB) for the router against 119.14 $\pm$ 0.01 and 89.71 $\pm$ 0.00 for static mixing, so the effect is a few standard deviations, not orders of magnitude.

	train	valid	test
<i>PTB: each expert's own perplexity</i>			
KN3 trigram	34.7	178.1	165.1
lag-2 bigram	135.8	1,335.0	1,220.3
cache (512 tokens)	1,739.4	1,780.2	1,615.7
lag-3 bigram	177.5	2,035.6	1,879.1
cont. unigram	1,048.6	1,048.7	992.0
GRU	46.6	111.8	103.0
<i>PTB: mixtures</i>			
router (train-trained)	20.6	146.8	138.2
its mean weights, static	29.8	112.3	105.4
uniform weights	46.4	144.5	138.5
static (valid-fit)	32.7	94.0	89.7

Table 4: The mechanism (router_diagnostic.py). Expert reliability is not stationary: every statistical expert is far stronger on the stream it was built from (KN3 34.7 $\rightarrow$ 165.1), while the GRU shifts much less (46.6 $\rightarrow$ 103.0). A train-fit router learns per-context trust in experts that stop being reliable; using its *own average* weights statically already recovers most of the damage (138.2 $\rightarrow$ 105.4), so the per-context modulation is what mis-transfers.

8 What This Says About Routing at Scale

The reason to care about routing at this footprint is the same reason to care about it at any footprint: spend the expensive component only where it pays. Three preconditions had to be met here before that question could even be asked, and all three are cheap to check at any scale.

- *Normalization.* A router that sees the candidate’s own scores produces a score, not a distribution (§4). One line settles it: $\sum_w P(w | \text{ctx})$.
- *Budget-matched baselines.* A tuned static mixture of the same experts is a far stronger opponent than a single tuned λ (119.1 vs. 135.8 on WT-2, 89.7 vs. 92.7 on PTB). Most of what looks like “routing wins” is “six experts beat two”.
- *Stationarity.* The router must be tuned on a stream whose expert-reliability profile matches deployment, or its per-context trust mis-transfers (§6). This is the one that costs 57–113% perplexity when ignored.

The cost side does not vanish at scale either: routing is 9–21% of per-token latency here for 1–2% perplexity, and a skip rule built on the router’s own weight for the expensive expert found almost nothing to skip (99.0%/89.7% usage at $\tau = 0.5$), be-

cause an honestly tuned router wants the strong expert nearly everywhere. Whether that changes when the expensive expert is a transformer layer — where the cost asymmetry is orders of magnitude larger and the cheap experts are relatively much weaker — is the question this study is meant to inform. We deliberately do not extrapolate to it.

9 Conclusion

At the bottom of the footprint range, learned per-token routing over frozen classical experts plus one small GRU buys a few percent over a tuned static mixture of the same experts, and a double-digit percent over classical two-expert interpolation — provided the router is tuned on held-out data, and provided its features are causal. Trained on the training stream it is worse than uniform mixing. The larger lesson is methodological: a gate that reads the scored word’s own probabilities produces an unnormalized “language model” whose perplexity is deflated by the vocabulary mass, and that mass was 2.56–2.66 in our own first run — enough to manufacture a 27–33% win out of nothing, complete with three seeds, two corpora, an ensemble and a size sweep that all agreed. Sum-to-one is a one-line check. Run it.

The cost side is equally unflattering: routing adds 0.46–0.47 ms/token in situ over statically mixing the same six experts (+10% on WT-2, +78% on PTB, where the GRU is cheap), for a 1–2% perplexity gain. At this footprint the deployed answer is the tuned static hybrid; the router is a research result about where per-token decisions do and do not pay.

References

- Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. 2003. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155.
- Nicolò Cesa-Bianchi and Gábor Lugosi. 2006. *Prediction, Learning, and Games*. Cambridge University Press.
- Shuo Chang, Anmol Lahiri, Isaac Alphonso, Barlas Oguz, and Mikhail Levit. 2015. [Discriminative training of context-dependent language model scaling factors and interpolation weights](#). In *IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU)*.
- Stanley F. Chen and Joshua Goodman. 1999. An empirical study of smoothing methods for language models. *Computational Linguistics*, 25(1):73–100.

Yoav Freund and Robert E. Schapire. 1997. [A decision-theoretic generalization of on-line learning and an application to boosting](#). *Journal of Computer and System Sciences*, 55(1):119–139.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. [Adaptive mixtures of local experts](#). *Neural Computation*, 3(1):79–87.

Frederick Jelinek. 1980. Interpolated estimation of Markov source parameters from sparse data. In Edzard S. Gelsema and Laveen N. Kanal, editors, *Pattern Recognition in Practice*, pages 381–397. North-Holland, Amsterdam.

Michael I. Jordan and Robert A. Jacobs. 1994. [Hierarchical mixtures of experts and the EM algorithm](#). *Neural Computation*, 6(2):181–214.

Reinhard Kneser and Hermann Ney. 1995. [Improved backing-off for M-gram language modeling](#). In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.

Puneet Mathur, Zhe Liu, Ke Li, Yingyi Ma, and Gil Keren. 2023. [PersonaLM: Language model personalization via domain-distributed span aggregated K-nearest neighbor retrieval](#). In *Findings of the Association for Computational Linguistics: EMNLP*.

Tomas Mikolov, Anoop Deoras, Daniel Povey, Lukáš Burget, and Jan Černocký. 2011. [Strategies for training large scale neural network language models](#). In *IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU)*.

Tomas Mikolov, Martin Karafiát, Lukáš Burget, Jan Černocký, and Sanjeev Khudanpur. 2010. Recurrent neural network based language model. In *Inter-speech*.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*. ArXiv:1701.06538.

Martin Sundermeyer, Ralf Schlüter, and Hermann Ney. 2012. [LSTM neural networks for language modeling](#). In *Interspeech*.

A Reproduction

`bash scripts/run-paper-pipeline.sh {wt2|ptb}` regenerates every router-dependent number for one corpus: causal feature collection, the leak guard’s unit tests, the main results table with the size sweep, the seed audit, selective activation, the reliability diagnostic, the cost harness, both normalization audits (current and v1), and the footprint metadata. The GRU expert and its aligned scores are inputs (`nn_`

`expert.py--score-only`); the KenLM and 5-gram baselines are router-independent (`benchmarks/kenlm_baseline.py`, `benchmarks/fix_5gram.py`).

`HEDGE_RESULTS` switches every artifact path between corpora.

B The Invalidated v1 Numbers

For the record, the v1 protocol (router features `[0:5]` = the scored word’s expert log-probabilities, `[12]` = its cache count; static mixture fit on train; “oracle” = per-position best expert) produced, on 217,004 / 75,623 aligned positions: 5-expert router 99.7 / 72.7; 6-expert router 90.3 / 68.0 ($h = 128 / h = 24$, three-seed mean $101.1 \pm 21.2 / 74.6 \pm 10.1$, ensembles 90.2 / 69.9); valid-trained router $81.5 \pm 4.1 / 70.7 \pm 12.1$; static mixture 171.0 / 101.7; tuned interpolation 135.8 / 92.7; per-position best expert 57.4 / 44.4. Every router row is invalid (unnormalized, $Z \approx 2.6$); the per-position-best row is unnormalized and was never a bound; the static row was fit on train while its competitors were tuned on validation (valid-fit: 119.1 / 89.7). Only the tuned-interpolation row survives unchanged, which is precisely why it looked like a baseline being beaten by a third. The artifacts, the logs, and the code that produced them (commit 9d2eac4) are kept under `benchmarks/results/*/legacy_leaky_v1/` with a README stating exactly this, and `benchmarks/normalization_audit.py--legacy` re-measures the deflation from them.